DP Computer Science: B 2.1.1 Variables and Data Types

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define and differentiate between **global (static)** and **local variables** in Java
- Identify and declare variables of the following data types: **int, double, boolean, char, and String**
- Construct simple Java programs that correctly utilise both global and local variables
- **Trace** the execution of a program to determine the value of variables at different points

Key Vocabulary

Term	Definition
Variable	A named storage location in memory that holds a value
Data type	Defines the kind of data a variable can store (e.g., int, double, boolean, char, String)
Global variable	A variable declared outside of any method; accessible throughout the class
Local variable	A variable declared inside a method; only accessible within that method
Scope	The region of a program where a variable is accessible
Static	A keyword in Java used to declare class-level (global) variables
Declaration	The process of creating a variable and specifying its data type
Initialisation	The process of assigning an initial value to a variable

Approaches to Learning (ATL) Elements

Skill	Description
Thinking (Critical Thinking)	Analysing and evaluating code to identify variable scope and trace program flow
Self-Management (Organisation)	Structuring code logically and managing program state through variables

2. Introduction: Variables as Memory

The Starter Question

> *"Imagine a program that calculates a user's age and also keeps a running total of how many users have entered their age. What different pieces of information would this program need to remember?"*

Why Variables Matter

```text
┌─────────────────────────────────────────────────────────────────┐
│              WHY VARIABLES MATTER                               │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              STORING DATA                             │  │
│    │                                                         │  │
│    │   Variables hold data while the program runs          │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              MANIPULATING DATA                        │  │
│    │                                                         │  │
│    │   Variables can be changed and updated                │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              DIFFERENT TYPES OF DATA                  │  │
│    │                                                         │  │
│    │   Different variables hold different types of data    │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

---

3. Data Types in Java

Core Data Types

Data Type	Description	Example	Memory Size
int	Whole numbers (positive, negative, zero)	`int age = 25;`	32 bits
double	Decimal numbers (floating point)	`double price = 19.99;`	64 bits
boolean	True or false values	`boolean isPassed = true;`	1 bit
char	Single character (in single quotes)	`char grade = 'A';`	16 bits
String	Sequence of characters (in double quotes)	`String name = "Alice";`	Variable

Variable Declaration and Initialisation

```java
// Declaration: Creating a variable
int age;
double price;
boolean isPassed;
char grade;
String name;

// Initialisation: Assigning a value
age = 25;
price = 19.99;
isPassed = true;
grade = 'A';
```

```
name = "Alice";

// Declaration and Initialisation (combined)
int age = 25;
double price = 19.99;
boolean isPassed = true;
char grade = 'A';
String name = "Alice";
```

Data Types in Python

Data Type	Description	Example
int	Whole numbers	`age = 25`
float	Decimal numbers	`price = 19.99`
bool	True or false	`is_passed = True`
str	Sequence of characters	`name = "Alice"`

---

4. Variable Scope

What is Scope?

> Scope is the region of a program where a variable is accessible. It determines where a variable can be used.

Local Variables (Method-level)

> A local variable is declared inside a method and is only accessible within that method.

```java
public class Example {
    public void myMethod() {
        int x = 10;  // Local variable
        System.out.println(x);  // ✅ Accessible
    }

    public void anotherMethod() {
        System.out.println(x);  // ❌ NOT accessible (x is local to myMethod)
    }
}
```

Global Variables (Class-level)

> A global variable (static) is declared outside of any method and is accessible throughout the class.

```java
public class Example {
    static int globalCount = 0;  // Global variable

    public void myMethod() {
        globalCount++;  // ✅ Accessible
        System.out.println(globalCount);
    }
```

```java
    public void anotherMethod() {
        globalCount--;  // ✅ Accessible
        System.out.println(globalCount);
    }
}
```

Scope Comparison

```text
┌─────────────────────────────────────────────────────────────────┐
│              SCOPE COMPARISON                                   │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌───────────────────────────────────────────────────────┐    │
│    │              LOCAL VARIABLE                          │    │
│    │                                                       │    │
│    │   • Declared inside a method                         │    │
│    │   • Accessible ONLY inside that method              │    │
│    │   • Created when method is called                   │    │
│    │   • Destroyed when method ends                     │    │
│    │   • Analogy: A sticky note used for one task       │    │
│    └───────────────────────────────────────────────────────┘    │
│                                                                 │
│    ┌───────────────────────────────────────────────────────┐    │
│    │              GLOBAL VARIABLE                         │    │
│    │                                                       │    │
│    │   • Declared outside any method                     │    │
│    │   • Accessible throughout the class                 │    │
│    │   • Created when program starts                    │    │
│    │   • Destroyed when program ends                     │    │
│    │   • Analogy: A public scoreboard                    │    │
│    └───────────────────────────────────────────────────────┘    │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Java Example: Global vs. Local

```java
public class ScopeExample {
    // Global variable (static)
    static int globalCounter = 0;

    public static void main(String[] args) {
        // Local variable
        int localNumber = 5;

        System.out.println("Global Counter: " + globalCounter);
        System.out.println("Local Number: " + localNumber);

        updateCounters();
    }

    public static void updateCounters() {
        globalCounter++;  // ✅ Accessible
        // localNumber++;   // ❌ NOT accessible (local to main)
    }
}
```

Python Example: Global vs. Local

```python
# Global variable
global_counter = 0

def main():
    # Local variable
    local_number = 5

    print("Global Counter:", global_counter)
    print("Local Number:", local_number)

    update_counters()

def update_counters():
    global global_counter  # Required to modify global variable
    global_counter += 1    # ✅ Accessible
    # local_number += 1    # ❌ NOT accessible (local to main)

main()
```

---

5. Trace Tables

What is a Trace Table?

> A trace table is a table used to track changes in variable values as a program executes line by line.

Trace Table Example

```java
public class TraceExample {
    static int total = 0;

    public static void main(String[] args) {
        int a = 5;
        int b = 10;
        total = a + b;
        System.out.println(total);
    }
}
```

Completed Trace Table

Line	a	b	total	Output
1 (start)	-	-	0	-
2 (a = 5)	5	-	0	-
3 (b = 10)	5	10	0	-
4 (total = a + b)	5	10	15	-
5 (print total)	5	10	15	15

Output: 15

6. Activity: Variables and Trace Tables

Instructions

1. Work independently or in pairs
2. Construct a small program using global and local variables
3. Complete a trace table for the given program

Worksheet Exercise 1: Complete the Trace Table

```java
public class Activity {
    static int x = 0;

    public static void main(String[] args) {
        int y = 5;
        x = x + y;
        int z = x * 2;
        System.out.println("x = " + x + ", z = " + z);
        x = x + 10;
        System.out.println("x = " + x);
    }
}
```

Trace Table

Line	x	y	z	Output
1 (start)	0	-	-	-
2 (y = 5)	0	5	-	-
3 (x = x + y)	5	5	-	-
4 (z = x * 2)	5	5	10	-
5 (print)	5	5	10	x = 5, z = 10
6 (x = x + 10)	15	5	10	-
7 (print)	15	5	10	x = 15

Worksheet Exercise 2: Create Your Own Program

Task: Write a program that:

1. Has a global variable `count` initialised to 0
2. In `main`, declares a local variable `num` and reads a value from the user
3. Adds `num` to `count`
4. Prints both `num` and `count`

7. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Starter question; discuss different types of memory/variables
Data Types	10 min	Explain core data types; syntax for declaration and initialisation
Variable Scope	10 min	Define local and global variables; use analogies; code examples
Live Coding & Tracing	15 min	Teacher live-codes a program; demonstrate trace table
Student Activity	15 min	Students complete worksheet (construct program and trace table)
Plenary	5 min	Review answers; summarise key differences

---

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate ability to declare variables and use trace tables
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is the difference between a local and a global variable?
2. What keyword is used for global variables in Java?
3. What data type would you use to store a decimal number?
4. What is the purpose of a trace table?

---

9. Differentiation

Level	Strategy
Support	Provide a "cheat sheet" with data type syntax; use visual analogies
Challenge	Explore variable scope with nested loops and multiple methods

---

10. Resources

- **Presentation** (Slide Deck)
- **eBook** (B2.1.1 with examples)
- **Worksheet:** Variables and trace tables
- **Quizlet** (B2.1.1 vocabulary flashcards)

---

11. Summary: Key Takeaways

Concept	Key Point
Variable	Named storage location in memory
Data Types	int, double, boolean, char, String
Local Variable	Declared inside a method; accessible only within that method
Global Variable	Declared outside methods; accessible throughout the class (static)
Scope	The region where a variable is accessible
Trace Table	Tracks variable values as a program executes

```text
┌─────────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                               │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ✅ Variables store data                                     │
│    ✅ Data types define what kind of data                      │
│    ✅ Local variables = inside methods (sticky notes)          │
│    ✅ Global variables = outside methods (scoreboard)          │
│    ✅ Scope = where variables are accessible                   │
│    ✅ Trace tables track variable changes                      │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

> "*Variables are the building blocks of programs—they store data, and their scope determines where they can be used.*"